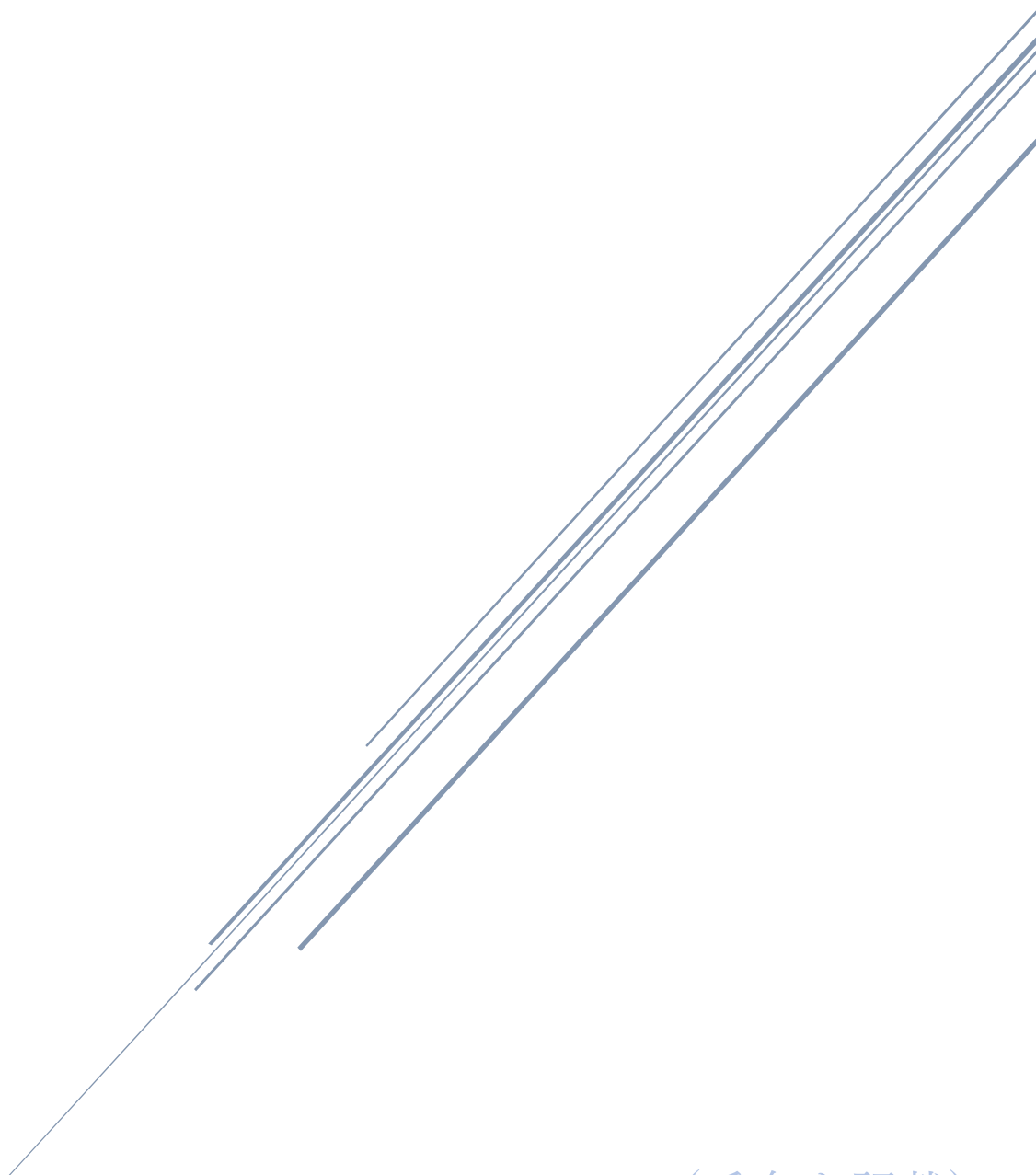


プログラムポートフォリオ



(氏名を記載)

目次

自己紹介.....	2
学習した技術.....	2
ゲームプレイ履歴.....	2
Tech Stadium について.....	3
作品・学習内容紹介.....	4
Torch.....	4
不動産管理 Web アプリケーション.....	1 3
Unity 公式チュートリアル『Tank』の改変.....	1 6
Java による 3DCG 演習.....	1 8
Python によるシラバス検索システム.....	1 9
終わりに.....	2 0

自己紹介

学習した技術

種類	期間	内容・作品
Unity・C#	一か月（2019/3）	・オリジナルゲーム一本の作成 ポートフォリオ内に記載
CakePHP・ phpMyAdmin	一か月（2019/3）	・不動産管理システム Web アプリケーションの作成 ・上記 Unity ゲームにおける API の作成 2点ともポートフォリオに記載
Python	一か月半（2018/10 頃）	大学の授業（演習）で履修 ・大学シラバスのキーワード検索システム ポートフォリオに記載
Java	三か月半（2015/4~7 上旬）	大学の授業（演習）で履修 ・スクリプトによる 3DCG ポートフォリオに記載
Ruby	三か月半（2014/9 下旬~12）	大学の授業『基礎情報学』で使用。
C++	学習中（2019/4~）	『独習 C++』などを参考に、2 ヶ月以内のオリジナル ゲーム制作を目指して学習中です。

ゲームプレイ履歴（特に関心があったものを抜粋）

コンシューマーゲーム

- ・『ゼルダの伝説 ブレスオブザワイルド』（Nintendo Switch / Nintendo）最も好きなゲーム。
フィールドとユーザーの視点誘導などの関係性が丁寧で、デザインの重要性を感じました。
- ・『風ノ旅ビト』（PS4 / SCE）ソーシャル要素と音響が魅力的でした。

スマートフォンゲーム

- ・『崩壊 3rd』（iOS / 株式会社 miHoYo）2018/～ 現在 Lv.67 ほど。微課金でコツコツ続けています。
- ・『スキタイのムスメ：音響的冒険劇』（iOS / Cappybara Games Inc.）音響とグラフィックが魅力的でした。

PC ゲーム

- ・『Portal』シリーズ（Steam / Valve Corporation）ゲームシステムと SF 世界観に引き込まれました。

Tech Stadium について

株式会社 LYUS 社運営の Tech Stadium 「Unity コース」を受講しました。

期間：2019年3月1日（金）～2019年3月29日（金）

カリキュラム：

1. クライアント技術学習(unity)

- ・ Git について
- ・ Unity 基礎/基本操作
- ・ UI (uGUI を利用した UI 作成)
- ・ Camera 制御
- ・ Audio 制御
- ・ ParticleSystem
- ・ 接触判定(Collision,Trigger)

2. サーバー技術学習(CakePHP)

- ・ ゲームサーバー基礎/サーバー構成
- ・ LAMP 環境構築(PHP,MySQL)
- ・ フレームワーク利用(CakePHP を利用した MVC、ORM)
- ・ DB 基礎(データベース/テーブル作成、query 制御)
- ・ Web ページ作成
- ・ API 学習/作成

3. クライアント、サーバー連携技術学習(unity & CakePHP)

- ・ Unity でのデータアクセス/ネットワークアクセス
- ・ CakePHP での API 作成、Unity との繋ぎこみ

4. その他

- ・ マルチプレイ(Photon)
- ・ 開発 Tips (プログラム、ツール、データベース、等)

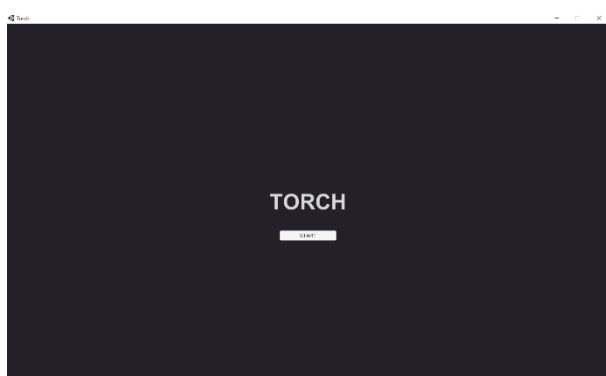
作品・学習内容紹介

Torch

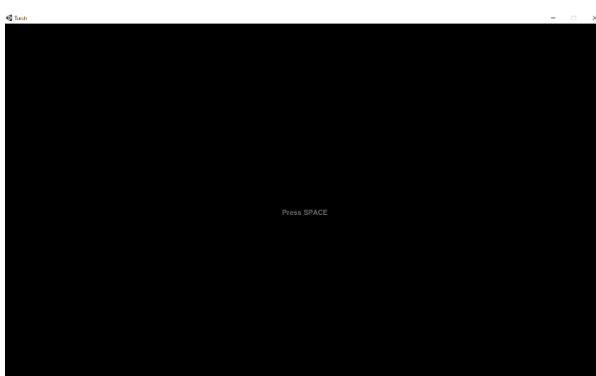
Git リポジトリ

<https://github.com/KensukeMori/torch.git>

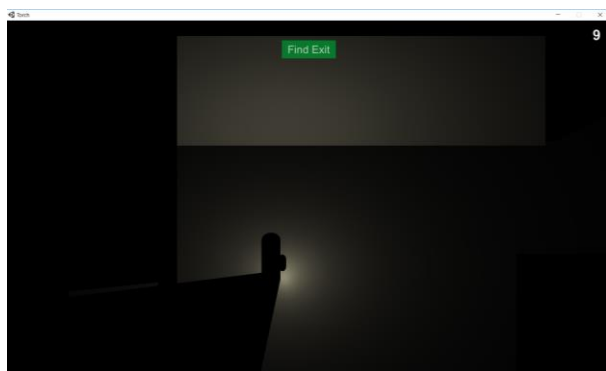
ゲームの画像



(左上：タイトル画面。Start ボタンを押下すると Stage 画面に遷移。)



(右上：スタート直後の画面。真っ暗なので「Press SPACE」の指示文を表示。)



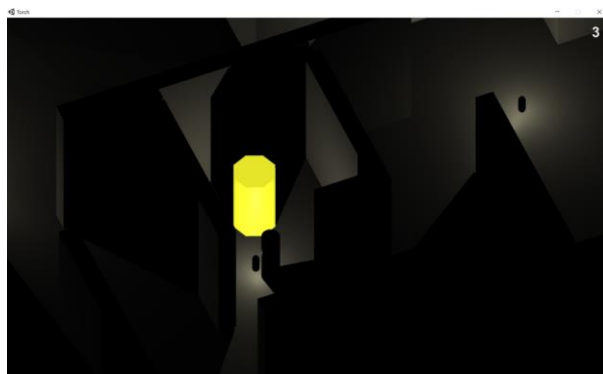
(左上：最初の松明を設置すると画面が明るくなり、「Find Exit」の指示（緑色）を3秒間表示。)



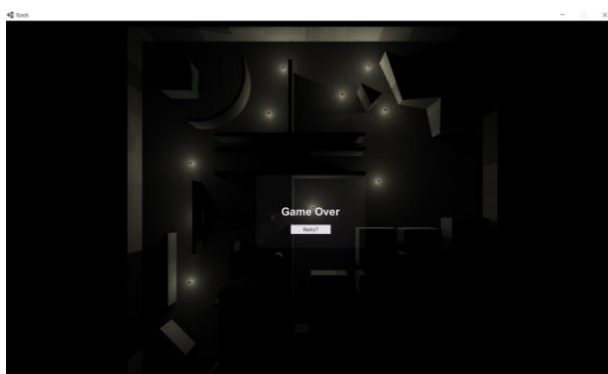
(右上：プレイ画面。右上のカウン트가設置できる残りの松明数。)



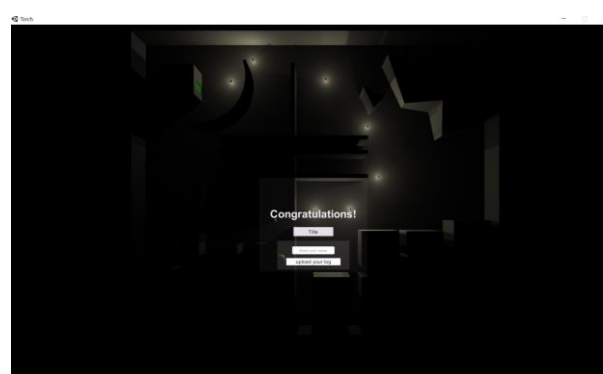
(左上：緑色の出口に接触するとゲームクリア画面に遷移。)



(右上：スイッチギミックに触れると画像のように発光する。ゴースト再生機能（後述）を起動する。)



(左上；ゲームオーバー画面。俯瞰ビューに切り替え経路がわかる。リトライボタンで再挑戦できる。)



(右上：ゲームクリア画面。松明設置ログのアップロードとタイトル画面への遷移ができる。)

ゲーム概要

1 基本説明

- i. 暗闇の中で松明をセットし、出口を探すゲームです。松明を生成するたびに UI のカウントが減少し、10本の松明を使い切るとゲームオーバーになります。
- ii. 松明をセットしたとき、ゲームオーバーになったとき、出口にたどり着いたときのそれぞれについて、①位置、②スタートからの経過時間、の情報がユーザー情報とともにゲーム終了時まで PlayerPrefs に保存されます。クリア画面でユーザー名を入力し、データベースに登録することができます。
- iii. サーバーに保存された情報はゲーム開始時に数件呼び出され、スイッチギミックに使用されます。スイッチギミックに接触すると、過去のユーザーのプレイログを再生し、松明が同じ時間ス

ケールで自動生成されます。ログが出口に到達する情報を保持していた場合、そのタイミングでその出口が使用不可に設定されます。

- iv. 基本的なゲーム性としては、必要数より少ない松明を使い切ることを繰り返しながら出口を探していくゲームとなっています。出口はスイッチの数より一つ以上多く設置されることとしていますが、スイッチを使うことでログのゴーストに先回りをしてゴールを探す競争要素と、松明を増やせる一方で出口を減らしてしまうリスクも生まれてしまうトレードオフの関係を取り入れました。

2 クリア条件等

- i. 出口の扉（緑）に接触するとゲームクリア
- ii. 松明の制限数を使い切り、右上のカウントが0になるとゲームオーバー

3 操作説明

- i. 移動：↑→↓←キー
- ii. Torch のセット：space キー

4 制作環境

- i. Unity 2017.1.3p4
- ii. XAMPP ver3.2.2 / PHP ver7.2.14 /MySQL ver15.1
- iii. CakePHP 3, phpMyAdmin

制作期間

2019年3月20日（水）～2019年3月29日（金）の10日間

技術面

全体構造

シーンは Title シーンとステージシーンのみとなっていて、ゲームクリア・オーバー時の画面は UI をアクティブ化しています。全体としては、PlayerController がほとんどの処理を担っており、移動や松明の生成、データログの取得、保存のトリガーとなります。そのほかには、GameManager が OnClick 系

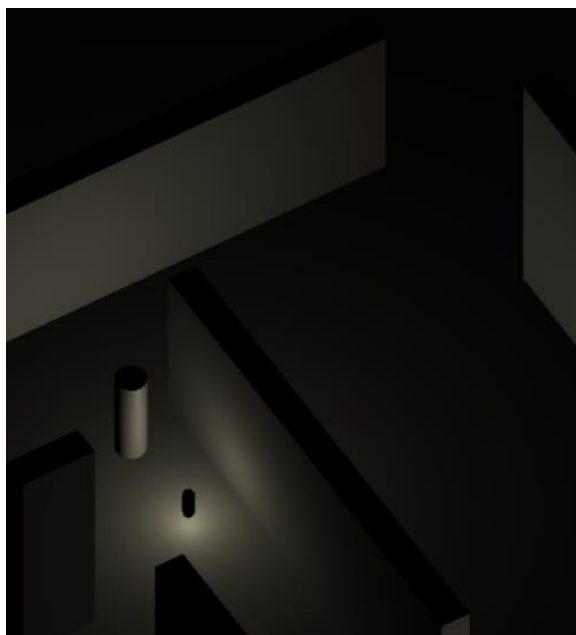
の処理を担い、出口やスイッチのマルチ化に対応してそれぞれ ExitManager, SwitchManager が Start 時に子オブジェクトの参照を行い、初期化を行います。

また、後述しますが、ItemData, SequenceData などのデータクラスを設定し、API の受け口として ItemDataModel, ItemDataManager クラスを設定しました。

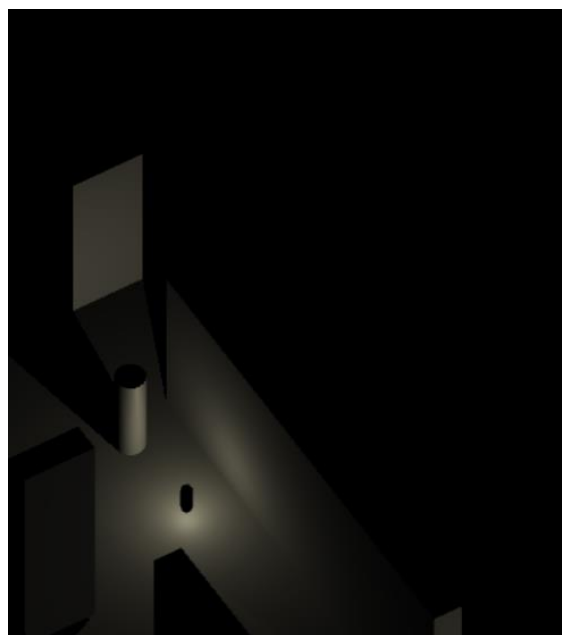
特に注意した部分

1. ライティング

通常の設定だとライトがなくても暗闇にならないことを経緯に、環境光などのライトの設定箇所について学習しました。また、松明のインスタンスとしてポイントライトを生成しつづけると途中からライトが光らないなど処理が安定しないことがあり、そのことを経緯としてベイクとリアルタイムの違いなど Unity のリファレンスを学習し、表現意図（ライトの直接光が当たっていない場所が完全な暗闇になる）にあった設定ができたと思います。



（左図：ライティング設定を行う前）



（右図：ライティング設定を行った後）

2. プレイログ保存・ゴースト機能

データについて

スイッチギミックに反応して再生されるゴースト機能の実装のため、PlayerPrefs とデータベースを活用しています。

データの保存のためにまず Item というデータクラスを作成し（下図）、座標・設置時間・どのステージか（現段階ではチュートリアルステージのみ）・何週目の挑戦かなどの情報を記録します。また、item データのリストとして sequence データクラスを作成しています。

```

1
2 //松明、デッドポイント、使用した出口の情報を保存するデータクラス
3 public class ItemData {
4
5     public int itemId;
6     public int itemType; //松明、デッドポイント、出口の種別
7     public int userId;
8     public int sequenceId;
9     public int stageId;
10    public float time;
11    public float itemXCoordinate;
12    public float itemYCoordinate;
13    public float itemZCoordinate;
14
15    public ItemData()
16    {
17        itemId = 0;
18        itemType = 0;
19        userId = 0;
20        sequenceId = 0;
21        stageId = 0;
22        time = 0f;
23
24        itemXCoordinate = 0f;
25        itemYCoordinate = 0f;
26        itemZCoordinate = 0f;
27    }
28 }
29
30
31

```

また、データベースには item テーブルを中心に 4 テーブルを作成し正規化しました。（下図 2 点）

テーブル	操作	行	データ型	照合順序	サイズ	オーバーヘッド
item	表示 構造 検索 挿入 空にする 削除	85	InnoDB	utf8_general_ci	16 KiB	-
itemtype	表示 構造 検索 挿入 空にする 削除	4	InnoDB	utf8_general_ci	16 KiB	-
stage	表示 構造 検索 挿入 空にする 削除	3	InnoDB	utf8_general_ci	16 KiB	-
user	表示 構造 検索 挿入 空にする 削除	4	InnoDB	utf8_general_ci	16 KiB	-
4 テーブル 合計		96	InnoDB	utf8_general_ci	64 KiB	0 バイト

#	名前	データ型	照合順序	属性	NULL	デフォルト値	コメント	その他	操作
☐	1	ItemId	int(11)		いいえ	なし		AUTO_INCREMENT	変更 削除 その他
☐	2	ItemType	int(11)		いいえ	なし			変更 削除 その他
☐	3	UserId	int(11)		いいえ	なし			変更 削除 その他
☐	4	SequenceId	int(11)		いいえ	なし			変更 削除 その他
☐	5	stageId	int(11)		いいえ	なし			変更 削除 その他
☐	6	Time	double(4,1)		いいえ	3.0			変更 削除 その他
☐	7	XCoordinate	double(5,2)		いいえ	なし			変更 削除 その他
☐	8	YCoordinate	double(5,2)		いいえ	0.00			変更 削除 その他
☐	9	ZCoordinate	double(5,2)		いいえ	なし			変更 削除 その他

サーバー側では API を作成し、Json 形式でアイテムのデータをやり取りします。

データのセット

Unity クライアント側では、松明のセットを管理している PlayerContoroller に List<item>をメンバー変数として持たせ、松明のセット時にログを保存します。ゲームオーバー判定も松明セット時に行うため、同様のタイミングでログにセットされます。ゴールの判定は Player 側の OnTriggerEnter で行うので、引数の接触対象から対象の出口番号を入手し、同様に記録します。

メンバー変数に保存したログは、ゲームオーバー時に 1 シークエンス分として PlayerPrefs に保存され、ゲームクリア時に全てのシークエンスをユーザーネームとともにサーバーにアップロードすることができます。どちらの場合も、保存・送信の前にリスト型から json 形式にシリアルライズし、string として処理を行います。

```

//アイテムを指定したアイテムリストに格納する関数
public void AddLog(List<ItemData> itemDataList, int itemtype)
{
    ItemData sitemData = new ItemData();
    sitemData.itemType = itemtype;
    sitemData.sequenceId = m_sequenceId;
    sitemData.stageId = 1; //とりあえずtestステージを指定
    sitemData.time = m_TimeFromStart;

    sitemData.itemXCoordinate = transform.position.x + m_TorchPosition.x;
    sitemData.itemYCoordinate = 0f;
    sitemData.itemZCoordinate = transform.position.z + m_TorchPosition.z;
    itemDataList.Add(sitemData);
}

//各シーケンス終了時にシリアルライズとPlayerPrefsへの格納を行う関数
public void SaveToPlayerPrefs() {
    PlayerPrefs.SetInt("sequenceId", m_sequenceId);

    ItemDataModel itemDataModel = new ItemDataModel();
    string prefsDic = itemDataModel.SerializeToJson(m_ItemLogList);

    PlayerPrefs.SetString("seq" + m_sequenceId.ToString(), prefsDic);

    Debug.Log(PlayerPrefs.GetString("seq" + m_sequenceId.ToString()));
}

```

(上図：PlayerController 内のデータの設定を行う処理の一部)

データの再生

スイッチギミックにアタッチした SwitchController で処理を行います。Player 側 OnTriggerEnter の回数から、接触したスイッチの処理を呼びます。時間経過にしたがってコルーチンで松明の生成を行い、ログが出口の情報を持っていれば、出口に到達した時間で対象の出口の判定を使えなくします。また、使用不可になった出口は赤色のマテリアルに変更されます。

```

//ゴーストの再生を行う関数
public IEnumerator PlayGhost()
{
    float sPassedTime = 0f; //item.timeがスタート時からの時間経過のため、差分をとるための変数

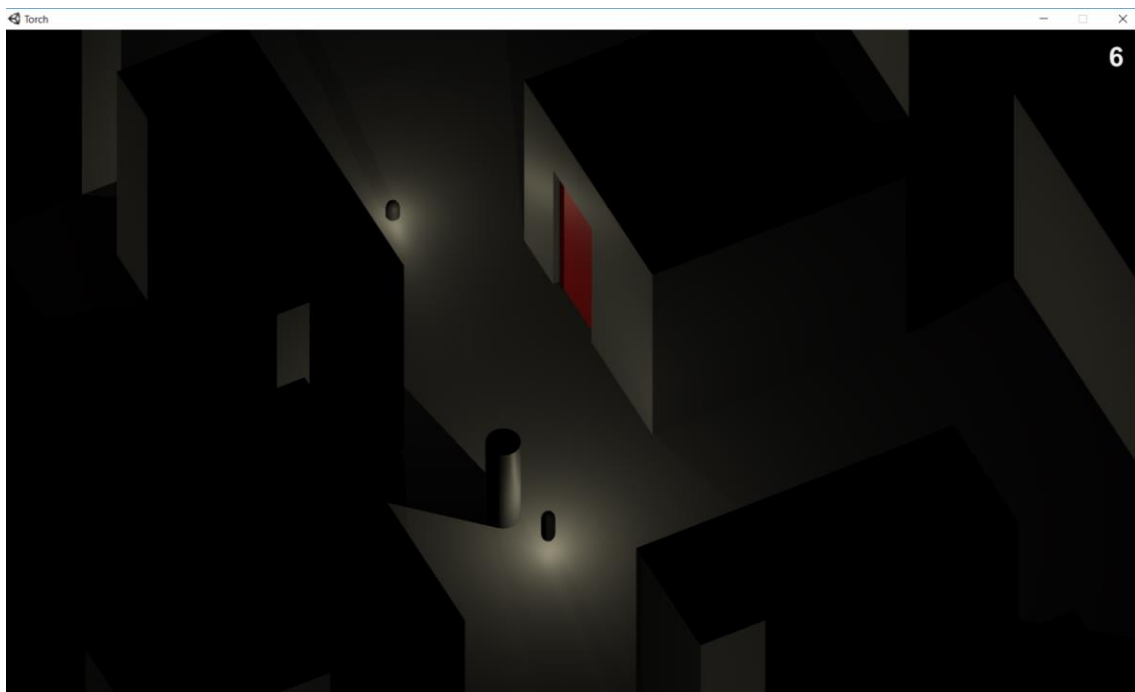
    foreach (ItemData item in m_itemList)
    {
        yield return new WaitForSeconds(item.time - sPassedTime);

        if(item.itemType == 1)
        {
            //torchをインスタンス化
            Vector3 sCoordinate = new Vector3(item.itemXCoordinate, item.itemYCoordinate, item.itemZCoordinate);
            Instantiate(m_TorchPrefab, sCoordinate, Quaternion.identity);
        }
        else if(item.itemType == 2)
        {
            //デッドポイント表示
            Debug.Log("deadpoint設置します。");
        }
        else if(item.itemType == 3)
        {
            //Exit封鎖
            Debug.Log("exit1封鎖します。");
            m_ExitManager.CloseExit(0);
        }
        else if (item.itemType == 4)
        {
            //Exit封鎖
            Debug.Log("exit2封鎖します。");
            m_ExitManager.CloseExit(1);
        }
    }

    sPassedTime = item.time;
}

```

(上図：SwitchController 内、ゴーストの再生を行う関数)



(上図：画面左上から到達したゴーストによって使用不可になった出口)

3. 拡張性

今回のデザイン・システム設計に共通して、拡張性の高いものを作成したいという気持ちがありました。移動して松明をセットするという基本的なシステムをきわめて簡略にし、機能やステージを拡張することでゲームとして豊かにでき、技術の実装実験ができるような設計をめざしました。

そのため若干コードやデータクラスの設計に使用されていない項目や冗長性があるかと思いますが、拡張機能を付ける際になるべくシステムのベースを変更しなくてもいいように余裕のある設計を目指したつもりです。

個人的な興味として、以下の機能を実装することを目指しています。

- 明るさの情報を感知し、暗闇にプレイヤーが入ると体力が減少する機能
- 明るさに反応する敵 AI
- CPU の作成・または photon を活用したマルチプレイ
- ログ機能を活用した強化学習によるマップの最適化
- プロシージャルによるマップ自動生成

不動産管理 Web アプリケーション

Git レポジトリ

<https://github.com/KensukeMori/EstateManager.git>

サイト画像

The screenshot shows a web browser window with the address bar displaying 'localhost/estateManagementSystem/room/search'. The page has a dark teal header with the word 'Room' in white. Below the header, the title '不動産検索システム' (Real Estate Search System) is displayed in a large, bold font. Underneath the title, the text '絞り込み' (Filtering) is shown. The form contains several input fields and checkboxes:

- '賃料の上限 (万円)' (Maximum Rent (in 10,000 yen)) with a value of '20' entered.
- '賃料の下限 (万円)' (Minimum Rent (in 10,000 yen)) with a value of '5' entered.
- '部屋の大きさ' (Room Size) section with two input fields: '上限 (㎡)' (Maximum in sqm) and '下限 (㎡)' (Minimum in sqm), with '20' entered in the latter.
- 'ルームタイプ' (Room Type) section with a list of checkboxes: 'Room Type', '1R', '1K', '1DK', '1LDK', '2LDK' (checked), '3LDK' (checked), and '4LDK'.
- '最寄り駅' (Nearest Station) section with a list of checkboxes: 'Station', '東京' (Tokyo), '神田' (Kojima), and '御茶ノ水' (Ochanomizu).

概要

Tech Stadium Unity コースにおける Web アプリケーション課題で制作。

不動産情報が登録されたデータベースから、

- 1 賃料の上限・下限
- 2 部屋の大きさの上限・下限
- 3 部屋のタイプ（複数指定 / ex. 1 K, 3 LDK）
- 4 最寄り駅（複数指定）

を指定して物件を検索します。検索結果は「賃料」または「部屋の大きさ」について昇順または降順で並び替えて表示することができます。

制作環境

- i. XAMPP ver3.2.2 / PHP ver7.2.14 /MySQL ver15.1
- ii. CakePHP 3, phpMyAdmin

技術面で特に注意した点

データベースの正規化

リレーショナルデータベースにおける正規化の概念を学習し、検索に用いたい部屋データを格納した room テーブルをメインに、room テーブルに紐づけるテーブルを 4 つ作成しました。

テーブル	操作	行	データ型	照合順序	サイズ	オーバーヘッド
☐ building	★ 表示 構造 検索 挿入 空にする 削除	5	InnoDB	utf8_general_ci	16 KiB	-
☐ resident	★ 表示 構造 検索 挿入 空にする 削除	7	InnoDB	utf8_general_ci	16 KiB	-
☐ room	★ 表示 構造 検索 挿入 空にする 削除	7	InnoDB	utf8_general_ci	16 KiB	-
☐ roomtype	★ 表示 構造 検索 挿入 空にする 削除	7	InnoDB	utf8_general_ci	16 KiB	-
☐ station	★ 表示 構造 検索 挿入 空にする 削除	32	InnoDB	utf8_general_ci	16 KiB	-
5 テーブル 合計		58	InnoDB	utf8_general_ci	80 KiB	0 バイト

(上図：estate データベース内構成)

#	名前	データ型	照合順序	属性	NULL	デフォルト値	コメント	その他	操作
☐ 1	roomId	int(11)			いいえ	なし		AUTO_INCREMENT	変更 削除 その他
☐ 2	roomName	int(11)			いいえ	なし			変更 削除 その他
☐ 3	building	int(11)			いいえ	なし			変更 削除 その他
☐ 4	roomSize	float			いいえ	なし			変更 削除 その他
☐ 5	roomType	int(11)			いいえ	なし			変更 削除 その他
☐ 6	rent	float			いいえ	なし			変更 削除 その他
☐ 7	resident	int(11)			いいえ	なし			変更 削除 その他

(上図：room テーブルの構造)

複数のテーブルを連結したクエリ操作

正規化に伴い、特に room テーブルから building テーブル内のレコードに検索をかける必要があったため、連結の概念を学習し実践しました。

```

public function result(){

    //モデルをロードし、他コントローラのテーブルも使えるようにする
    $this->loadModel('Building');
    $this->loadModel('Station');
    $this->loadModel('Roomtype');

    //データベースからデータを取得
    $roomQuery = $this->Room->find('all')
        ->join([
            'table' => 'Building',
            'alias' => 'bld',
            'type' => 'LEFT',
            'conditions' => 'bld.buildingId = Room.building'
        ])->select([
            'roomId' => 'Room.roomId',
            'roomName' => 'Room.roomName',
            'building' => 'Room.building',
            'roomSize' => 'Room.roomSize',
            'roomType' => 'Room.roomType',
            'rent' => 'Room.rent',
            'resident' => 'Room.resident',
            'stationId' => 'bld.nearbyStation'
        ]);
}

```

(上図：Room コントローラー内 result アクションにおける join 操作)

データの扱い

クライアント側に結果データを送る前に必要なデータのみが送信されるような整形を行い、通信が不必要に増大することがないように留意しました。

```

284
285 //Viewに引き渡したいデータのみを表示名で挿入
286 $eachBuilding['building'] = array(
287     'buildingName' => $buildingQueryCloned[0]['buildingName'],
288     'nearbyStation' => $stationName[$buildingQueryCloned[0]['nearbyStation'] - 1],
289     'distance' => $buildingQueryCloned[0]['distance']
290 );
291
292 $eachBuilding['rooms'] = array();
293 }
294 //部屋情報を表示名で配列に追加
295 $eachBuilding['rooms'][] = array(
296     'roomName' => $room['roomName'],
297     'rent' => $room['rent'],
298     'roomType' => $roomType[$room['roomType'] - 1],
299     'roomSize' => $room['roomSize']
300 );

```

(上図：RoomController result アクション内で引き渡すデータを整えている箇所)

Unity 公式チュートリアル『Tank』の改変

画像



(左図：バリア体力フル時の様子)



(右図：被弾時にバリア体力が減少する様子)

概要

Unity 公式チュートリアルに従って Tank プロジェクトを作成し、バリア機能を実装しました。左右シフトを押下している間バリアが発動し、バリアに砲弾が接触すると戦車の代わりにバリアの体力が減少します。バリアの体力は戦車の周囲にスライダー表示（外側の円）されており、バリアの体力が 0 になると一定期間バリアの起動ができなくなります。

またバリア展開時は砲弾の発射ができなくなる仕様と砲弾発射後の数フレーム間バリアが発動されないように調整を行うことでバリアが自弾によって被弾しないよう調整を行いました。

技術面

改変前の構造として、Player(Tank)側でイベント処理を行い、Shell などのインスタンスされたものについてはそれぞれのオブジェクトにアタッチされたスクリプトで処理を行っているものと認識したため、イベント処理を Tank にアタッチされた BarrierActivation で行い、Barrier オブジェクトにアタッチされた BarrierHealth 内で実際に被弾した際のスライダーの調整などの処理を行いました。

また、Tank 2 台についてそれぞれインプットの処理が異なるように設定することや、コルーチンを用いて体力回復や自弾の発射後の硬直などの処理を設定するなど細部に配慮しました。

```

27 //
28 public void Initialize ()
29 {
30     //現在のバリア体力値を初期値に設定
31     m_CurrentBarrierHealth = m_StartingBarrierHealth;
32     m_Broken = false; //バリアは破壊されていない
33
34     SetBarrierUI();
35     m_barrierObject.SetActive(false);
36 }
37
38
39 public void DamageToBarrier (float amount)
40 {
41     //バリアにamountだけのダメージを与える際の処理
42
43     m_CurrentBarrierHealth -= amount; //バリア体力から差し引き
44
45     SetBarrierUI(); // UIに反映
46
47     // ダメージによりバリア体力が0になった場合は破壊処理へ
48     if (m_CurrentBarrierHealth <= 0f && !m_Broken)
49     {
50         StartCoroutine(OnBroken());
51     }
52 }
53
54 private void SetBarrierUI()
55 {
56     //BarrierSliderにバリア体力値を同期
57     m_BarrierSlider.value = m_CurrentBarrierHealth;
58
59     //BarrierSliderの色をバリア体力値に合わせて設定
60     m_BarrierFillImage.color = Color.Lerp(m_ZeroBarrierHealthColor,
61     m_FullBarrierHealthColor, m_CurrentBarrierHealth / m_StartingBarrierHealth);
62 }
63
64
65 //バリアーが破壊された後にバリアを回復する操作
66 private IEnumerator BarrierRecovery()
67 {
68     m_BarrierRecoverSpeed = m_StartingBarrierHealth / m_RecoveringTime;
69     m_BarrierFillImage.color = Color.magenta;
70
71     while (m_CurrentBarrierHealth < m_StartingBarrierHealth)
72     {
73         m_CurrentBarrierHealth += m_BarrierRecoverSpeed * Time.deltaTime;
74         m_BarrierSlider.value = m_CurrentBarrierHealth;
75         yield return null;
76     }
77     m_CurrentBarrierHealth = m_StartingBarrierHealth;
78     SetBarrierUI();
79 }
80
81 //バリア体力がなくなった際の処理
82 private IEnumerator OnBroken ()
83 {
84     m_Broken = true; //破壊判定を真に変更
85
86     m_barrierObject.SetActive(false); //バリアを非アクティブに
87
88     m_CurrentBarrierHealth = 0f; //バリア体力が負か0なので念のため0にセット
89     SetBarrierUI(); //UIに反映
90
91     //バリアの回復処理を終えたら破壊判定解除
92     yield return StartCoroutine("BarrierRecovery");
93
94     m_Broken = false;
95 }
96
97

```

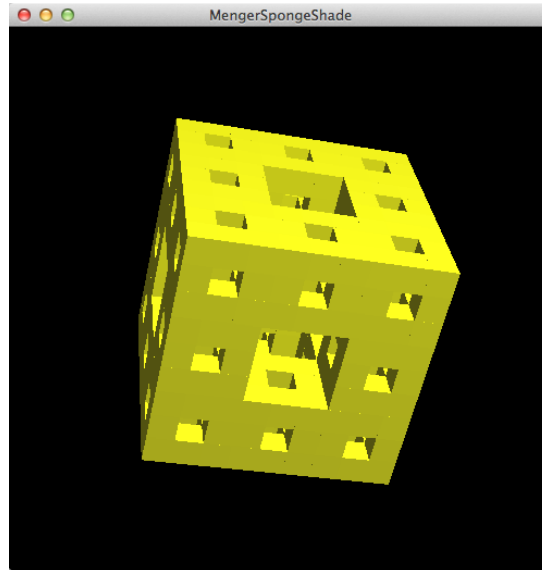
(上図2点：BarrierHealth スクリプトの一部)

制作期間

2019年3月前半の5日間ほど

Java による 3DCG 演習

画像



(図：制作事例。フラクタル立体のシェーディング表示。)

概要

Java を用いてグラフィックプログラミングの技術を学習しました。授業は書籍『Java による 3DCG 入門』（著：山口泰 [amazon](https://www.amazon.co.jp/dp/4800421110)）に沿っており、一章ごとに予習し個人で章末の課題を作成する形式でした。

授業を通じて、マウスイベントなどのインタラクション、行列処理を用いたカメラビューや 3 次元立体の回転など 3D グラフィックの基礎を学習し、陰線表示やシェーダープログラミングなどについても学習することで、3DCG プログラミングの仕組みを体験することができました。

学習期間

2015 年 4 月～7 月初めまで大学の授業にて

Python によるシラバス検索システム

概要

JupyterNotebook を用いて、Python の基本文法と JSON, CSV などのファイルの取り扱い、Numpy などのライブラリの基本を学習し、機械学習の実践も行いました。最終課題として、Json ファイルで与えられた大学のシラバスについてキーワード類似度で並び替え、JupyterNotebook 上に表として表示するプログラムを作成しました。

本課題により、ファイルの形式を学び、条件などを用いて要求された形式に整形する経験が得られました。先述の Web アプリケーションの作成や API を用いた Unity ゲームの作成に役立っています。

学習期間

2018年9月～10月の6週間ほど、大学の授業にて

終わりに

私にとって 2019 年 3 月に思い切って Tech Stadium Unity コースを受講してからの生活はめまぐるしい変化の連続でした。C#はおろか Unity の使用経験すらない状態から、バージョン管理ツールやデバッグ機能の扱い方など IT の基礎リテラシーやクライアントからサーバーまで一通りのゲーム制作の基本を学び、オリジナルゲームを制作するに至りました。いわゆる詰込みともいえる状態で毎日起きてから寝るまでプログラムに向き合いましたが、毎日が楽しくて仕方ありませんでした。毎日新しい技術を学び、自分で動かし、エラーと向き合い解消しました。

このポートフォリオを完成させて、まだ自分には不足している技術が無数にあることを実感しました。まだまだ学びたい技術がありますし、挑戦したいことも数えきれずあります。そのことは自分にとっては苦痛ではなく、とても心躍ることです。自分の強みである吸収力と素直さを活かして、これからも日々学習しつづけ、周囲の方々と協調して、ゲーム業界に貢献できるエンジニアに成長していきたいと思います。

ここまで読んでいただきありがとうございました。

以上